

Advanced Certification in Agentic AI Engineering

Course Curriculum : Your 25 module Learning Plan

<https://www.edureka.co/agentic-ai-training-course>

About Edureka

Edureka is a leading e-learning platform providing live instructor-led interactive online training. We cater to professionals and students across the globe in categories like Big Data & Hadoop, Business Analytics, NoSQL Databases, Java & Mobile Technologies, System Engineering, Project Management and Programming. We have an easy and affordable learning solution that is accessible to millions of learners. With our students spread across countries like the US, India, UK, Canada, Singapore, Australia, Middle East, Brazil and many others, we have built a community of over 1 million learners across the globe.

Curriculum

Module 01: Python & AI Dev Environment Setup

Topics Covered

- Overview of the Agentic AI ecosystem and where Python fits
- Setting up Python with pyenv for version management
- Creating and managing virtual environments using venv and conda
- Configuring VS Code with AI-friendly extensions and settings
- Setting up Jupyter Lab for interactive AI experimentation
- Understanding Python project structure for production AI apps
- Managing dependencies with pip and requirements.txt
- Securely managing API keys using python-dotenv and .env files
- Introduction to Git: initialising repos, branching, and committing
- Pushing AI projects to GitHub with a proper .gitignore
- Writing asynchronous Python with asyncio for concurrent LLM calls
- Comparing sequential vs concurrent API calls: performance benchmarking

Hands-On

- Setting Up Your Python AI Project from Scratch
- Managing API Keys Securely with python-dotenv
- Running Concurrent LLM Calls with asyncio

Skills Covered

- Python
- Virtual Environments
- Git & GitHub
- asyncio

- Project Structure
- Secret Management

Module 02: FastAPI, Streamlit & Gradio for AI Apps

Topics Covered

- REST API fundamentals: HTTP methods, status codes, request/response lifecycle
- FastAPI project structure: routers, models, and dependency injection
- Defining Pydantic models for request validation and response schemas
- Building async FastAPI endpoints for LLM-powered routes
- Streaming LLM responses using Server-Sent Events (SSE)
- Handling file uploads in FastAPI for document processing pipelines
- Running and testing FastAPI with Uvicorn and Swagger UI
- Streamlit fundamentals: layout, widgets, session state, and caching
- Building a real-time streaming chat UI in Streamlit
- Gradio components: text, file, image inputs and markdown outputs
- Creating shareable Gradio demos and deploying to HuggingFace Spaces
- Wiring a Streamlit or Gradio front-end to a FastAPI backend
- Comparing Streamlit vs Gradio: when to use which

Hands-On

- Building Your First FastAPI LLM Endpoint
- Streaming AI Responses Live in a Streamlit UI
- Creating a Shareable AI Demo with Gradio
- Connecting a Gradio Front-End to a FastAPI Backend

Skills Covered

- FastAPI
- Streamlit
- Gradio
- REST API Design
- Server-Sent Events
- Uvicorn
- Pydantic

Module 03: LLM Fundamentals, Context & Prompt Engineering

Topics Covered

- How large language models work: tokenisation, attention, and next-token prediction
- Understanding context windows: limits, costs, and implications for agents
- The OpenAI API: chat completions, parameters, and response structure
- The Anthropic API: messages format, system prompts, and model differences
- Structuring system prompts for domain-expert personas
- Zero-shot, one-shot, and few-shot prompting patterns
- Chain-of-Thought (CoT) prompting for complex reasoning tasks
- Tree-of-Thought (ToT) for branching multi-path reasoning
- ReAct prompting: combining reasoning and action steps
- Structured outputs using JSON mode and response schemas
- Function calling: defining tools and routing user intent
- Context management strategies: chunking, summarisation, and windowing
- Measuring and optimising token usage with tiktoken
- Evaluating and benchmarking LLM output quality

Hands-On

- Comparing Zero-Shot, Few-Shot, and Chain-of-Thought Prompts
- Extracting Structured Data with JSON Mode and Pydantic
- Routing User Intent with Function Calling

Skills Covered

- Prompt Engineering
- OpenAI API
- Anthropic API
- Context Management
- Function Calling
- JSON Mode
- Token Optimisation

Module 04: Embeddings, Vector Databases & Semantic Search

Topics Covered

- What vector embeddings are and why they matter for AI agents
- Embedding models: OpenAI text-embedding-3 and HuggingFace sentence-transformers
- Generating embeddings for text, documents, and structured data
- ChromaDB architecture: collections, documents, metadata, and IDs
- CRUD operations and metadata filtering in ChromaDB
- FAISS index types: Flat L2, IVF, HNSW — trade-offs explained
- Building and querying a FAISS index from a document corpus
- Nearest-neighbour retrieval: understanding similarity metrics (cosine, L2, dot product)
- Hybrid search: combining BM25 keyword scoring with semantic vector scores
- Chunking strategies for effective embedding: fixed, sentence, and recursive
- Benchmarking ChromaDB vs FAISS: speed, accuracy, and scalability

- Visualising embedding clusters with UMAP to understand semantic groupings
- RAG architecture overview: retriever, generator, and the full pipeline

Hands-On

- Building a Semantic Search Engine with ChromaDB
- Benchmarking FAISS Flat vs HNSW Retrieval Speed
- Implementing Hybrid Keyword and Semantic Search
- Visualising Document Embeddings with UMAP

Skills Covered

- Vector Embeddings
- ChromaDB
- FAISS
- Semantic Search
- Hybrid Search
- sentence-transformers
- RAG Fundamentals

Module 05: LangChain Core - Chains, Memory & RAG

Topics Covered

- LangChain architecture: runnables, the pipe operator, and the LCEL paradigm
- Building chains with LCEL: prompt | llm | parser patterns
- Prompt templates: ChatPromptTemplate, MessagesPlaceholder, and partial templates
- Output parsers: StrOutputParser, JsonOutputParser, and PydanticOutputParser
- Conversational memory types: BufferMemory, SummaryMemory, VectorStoreMemory
- Maintaining multi-turn conversation history across sessions

- Document loaders: PDF, web, CSV, Notion, and custom loaders
- Text splitters: RecursiveCharacterTextSplitter and SemanticChunker
- Building a RAG pipeline: loader, splitter, embedder, retriever, generator
- Retriever types: VectorStoreRetriever, MultiQueryRetriever, ContextualCompressionRetriever
- LangChain callbacks and hooks for logging and monitoring
- Chaining multiple retrievers and combining results

Hands-On

- Building an LCEL Chain with Memory for Multi-Turn Conversations
- Creating a RAG Pipeline over a PDF Knowledge Base
- Comparing Multi-Query vs Single-Query Retrieval Quality

Skills Covered

- LangChain
- LCEL
- RAG Pipelines
- Conversational Memory
- Document Loading
- Text Splitting
- Retrieval Strategies

Module 06: LangChain Agents & Tool Use

Topics Covered

- Agent fundamentals: perception, reasoning, action, and observation loop
- ReAct agent pattern: interleaving reasoning traces and tool calls
- Plan-and-Execute pattern: generating a plan first, then executing steps

- LangChain built-in tools: Tavily Search, Python REPL, Wikipedia, and DuckDuckGo
- Creating custom tools with the @tool decorator and StructuredTool
- Defining tool input schemas with Pydantic for safe structured arguments
- AgentExecutor: configuration, max iterations, early stopping, and verbose mode
- Structured output agents: forcing final answers into typed Pydantic objects
- Connecting agents to SQL databases with SQLDatabaseToolkit
- Multi-step reasoning: chaining tool outputs as inputs to subsequent tool calls
- Handling tool failures: try/except patterns and automatic retry logic
- Streaming intermediate agent steps to a UI in real time

Hands-On

- Building a ReAct Agent with Web Search and Python REPL Tools
- Connecting an Agent to a SQL Database for Natural Language Queries
- Adding Retry Logic and Error Handling to a Tool-Calling Agent

Skills Covered

- LangChain Agents
- ReAct Pattern
- Custom Tool Development
- AgentExecutor
- SQL Agent
- Structured Output
- Tool Error Handling

Module 07: LangGraph - Stateful Workflows & Routing

Topics Covered

- Why LangGraph: limitations of linear chains for complex agentic workflows
- Core LangGraph concepts: nodes, edges, and the state schema
- Defining a TypedDict state and passing it through the graph
- Building a StateGraph: adding nodes, setting entry points, and compiling
- Conditional edges: routing to different nodes based on state values
- MessageGraph for chat-style workflows with message history
- Streaming node-by-node output during graph execution
- Adding human-readable labels and descriptions to graph nodes
- Visualising and debugging graphs in LangGraph Studio
- Building a specialist routing system: classifier node + expert nodes
- Handling terminal states: defining END conditions and exit paths
- Composing reusable node functions with clean state interfaces
- Testing individual nodes in isolation before wiring the full graph

Hands-On

- Building a Conditional Routing Graph with Specialist Nodes
- Streaming Live Node-by-Node Output in LangGraph
- Debugging and Visualising Your Graph in LangGraph Studio
- Building an Order-Processing State Machine with Fail and Retry States

Skills Covered

- LangGraph
- StateGraph
- Conditional Routing
- Graph Compilation
- LangGraph Studio
- Stateful Agent Design
- Node Streaming

Module 08: LangGraph - Cycles, HIL & Persistence

Topics Covered

- Cycles and loops: when and why to add feedback loops to a graph
- Implementing self-correction loops: generate, evaluate, revise
- Human-in-the-loop (HIL): understanding `interrupt_before` and `interrupt_after`
- Pausing a graph, collecting human input, and resuming from a checkpoint
- Building approval gate workflows: draft, review, approve, publish
- Checkpointers: what they are and why persistence matters for agents
- SQLite checkpointer: setup, configuration, and state recovery
- Redis checkpointer: setup for high-throughput production environments
- Resuming an interrupted workflow after a simulated crash
- The Send API: dispatching work to parallel worker nodes
- Aggregating parallel node results back into a shared state
- Sub-graphs: building modular graph components and composing them
- Sharing state across parent and sub-graphs safely

Hands-On

- Adding a Human Approval Gate with Interrupt and Resume
- Implementing SQLite Checkpointing and Crash Recovery
- Fanning Out Tasks to Parallel Nodes with the Send API

Skills Covered

- Human-in-the-Loop
- LangGraph Checkpointers
- SQLite Persistence
- Redis Persistence
- Parallel Execution

- Sub-graph Composition
- Cycle Design

Module 09 - LangSmith - Tracing, Evaluation & Testing

Topics Covered

- Why observability matters for LLM applications in production
- LangSmith platform overview: projects, runs, traces, and feedback
- Instrumenting LangChain and LangGraph runs with LangSmith tracing
- Inspecting individual spans: inputs, outputs, latency, and token cost
- Creating evaluation datasets: golden QA pairs and edge cases
- Running automated evaluations with built-in LangSmith evaluators
- LLM-as-judge evaluation: writing a custom grading prompt
- Heuristic evaluators: rule-based checks for format, length, and keywords
- Regression testing: comparing two prompt versions on the same dataset
- A/B testing agent behaviours: setting up experiments and tracking metrics
- Annotating runs with human feedback for RLHF dataset creation
- Production monitoring dashboards: latency, error rate, and cost over time
- Setting up alerts for anomalous agent behaviour in production

Hands-On

- Tracing a LangGraph Agent End-to-End in LangSmith
- Running an Automated Evaluation Suite with LLM-as-Judge
- Setting Up an A/B Experiment to Compare Two Agent Strategies

Skills Covered

- LangSmith

- LLM Tracing
- Automated Evaluation
- LLM-as-Judge
- Regression Testing
- A/B Testing
- Production Monitoring

Module 10: Model Context Protocol: Architecture & Custom Servers

Topics Covered

- What is the Model Context Protocol and why Anthropic created it
- MCP architecture: hosts, clients, and servers — roles and responsibilities
- The three MCP primitives: resources, tools, and prompts
- JSON-RPC 2.0 transport layer: stdio and HTTP/SSE communication
- Setting up the MCP Python SDK and scaffolding a minimal server
- Defining and exposing custom tools from an MCP server
- Defining resource endpoints that expose live or static data sources
- Implementing prompt templates as MCP primitives
- Tool discovery and capability negotiation between client and server
- Connecting a custom MCP server to Claude Desktop
- Connecting a custom MCP server to VS Code Copilot
- MCP security model: permission scopes and access control
- Server-side input validation and request sanitisation
- Debugging MCP servers with logging and the MCP Inspector tool

Hands-On

- Scaffolding and Connecting Your First Custom MCP Server

- Exposing a Live Data API as an MCP Resource Endpoint
- Implementing Permission Checks and Input Validation in MCP
- Debugging an MCP Server with the MCP Inspector Tool

Skills Covered

- Model Context Protocol
- MCP Python SDK
- Custom MCP Servers
- Claude Desktop Integration
- Tool Discovery
- MCP Security
- JSON-RPC

Module 11: MCP - Ecosystem Integrations

Topics Covered

- Overview of the MCP ecosystem: official and community servers
- SQLite MCP server: exposing schema, running queries, and returning results
- Filesystem MCP server: safe file read, write, list, and move operations
- GitHub MCP server: managing issues, PRs, branches, and comments via Claude
- Playwright MCP server: browser automation and structured data extraction
- Chaining multiple MCP servers in a single Claude session
- Using MCP servers as tools inside LangChain agents
- Using MCP servers as tools inside LangGraph nodes
- Integrating MCP into a CrewAI agent as an external tool
- Building a report pipeline: SQLite MCP query to Filesystem MCP write
- Security considerations when chaining multiple MCP servers

- Production deployment patterns for MCP servers

Hands-On Demonstrations

- Chaining SQLite MCP and Filesystem MCP to Build a Report Pipeline
- Managing a GitHub Repository via Claude and the GitHub MCP Server
- Automating Browser Data Extraction with Playwright MCP

Skills Covered

- SQLite MCP
- GitHub MCP
- Playwright MCP
- Filesystem MCP
- Multi-MCP Chaining
- MCP + LangGraph
- MCP + CrewAI

Module 12: Deep Agents - Reflection, Planning & Long-Term Memory

Topics Covered

- What makes an agent 'deep': meta-cognition and self-awareness in LLMs
- Self-reflection loops: generate, critique against a rubric, revise
- Multi-turn reflection: iterating until a quality threshold is met
- Plan-and-Execute pattern: separating planning from execution
- Replanning: updating the plan mid-execution when new information arrives
- ReAct vs Plan-and-Execute: trade-offs and when to use each
- Long-term memory types: episodic, semantic, and procedural
- Building an episodic memory store with LangGraph and a vector DB

- Memory consolidation: summarising and compressing old episodic memories
- Semantic memory: storing facts and updating them across sessions
- Cross-session memory retrieval: using similarity search to recall past context
- Confidence and uncertainty estimation: flagging low-confidence answers
- Agent self-improvement: using evaluation scores to update future behaviour
- Combining reflection, planning, and memory into one unified agent architecture

Hands-On Demonstrations

- Building a Self-Critique and Revision Loop in LangGraph
- Implementing a Plan-and-Execute Agent with Mid-Run Replanning
- Adding Cross-Session Episodic Memory to an Agent

Skills Covered

- Self-Reflection Loops
- Plan-and-Execute
- Episodic Memory
- Semantic Memory
- Memory Consolidation
- Confidence Estimation
- Deep Agent Architecture

Module 13: CrewAI - Multi-Agent Orchestration & Integration

Topics:

- Why Build Custom MCP Servers
- FastMCP Framework Overview
- Defining MCP Tools
- Defining MCP Resources

- Defining MCP Prompts
- Type Validation with Pydantic
- Error Handling in MCP Servers
- MCP Server Deployment Patterns
- Testing MCP Servers
- Building a Custom MCP Server from Scratch

Hands-on

- Building a Custom MCP Server from Scratch

Skills you will learn:

- FastMCP
- Custom MCP servers
- Pydantic validation
- Server testing

Module 14 - Agentic RAG & GraphRAG

Topics Covered

- RAG failure modes: hallucination, missed retrieval, and irrelevant context
- Diagnosing RAG failures systematically with RAGAS metrics
- Agentic RAG: the agent decides whether, what, and how to retrieve
- Self-RAG: retrieve, score chunk relevance, decide to regenerate, synthesise
- Corrective RAG (CRAG): falling back to web search when local KB fails
- Advanced chunking strategies: semantic chunking, late chunking, parent-child
- Re-ranking retrieved chunks with Cohere and cross-encoder models
- HyDE: generating a hypothetical answer to improve query embedding

- Query expansion: generating multiple reformulations to broaden recall
- GraphRAG: representing documents as a knowledge graph of entities and relations
- Building a knowledge graph from documents using NetworkX
- Multi-hop reasoning: traversing the graph to answer complex questions
- Hybrid RAG: combining graph traversal with vector similarity search
- Evaluating RAG pipelines with RAGAS: faithfulness, answer relevancy, context recall
- Benchmarking Agentic RAG vs standard RAG on the same test set

Hands-On Demonstrations

- Implementing Self-RAG with Relevance Scoring and Conditional Regeneration
- Building a Corrective RAG Pipeline with Web-Search Fallback
- Constructing a Knowledge Graph and Running Multi-Hop GraphRAG Queries
- Evaluating Your RAG Pipeline End-to-End with RAGAS

Skills Covered

- Agentic RAG
- Self-RAG
- Corrective RAG
- GraphRAG
- Knowledge Graphs
- Re-ranking
- HyDE
- RAGAS Evaluation

Module 15: Agentic Workflows with N8N

Topics Covered

- N8N overview: visual workflow automation and its role in agentic systems

- Self-hosting N8N with Docker and configuring for production
- N8N core concepts: workflows, nodes, triggers, and credentials
- Webhook trigger node: receiving external events to kick off workflows
- OpenAI node: prompt construction, model selection, and response parsing
- Anthropic node: connecting Claude to N8N workflows
- HTTP Request node: calling any external API from a workflow
- Slack node: sending formatted messages and alerts
- Error handling in N8N: error branches, retry logic, and fallback paths
- Cron trigger: scheduling agent workflows to run on a timetable
- N8N expressions: transforming data between nodes with JavaScript
- Integrating N8N with LangGraph via HTTP webhook nodes
- Storing workflow outputs: Google Sheets, Notion, and Airtable nodes
- Deploying N8N workflows to production and managing credentials securely

Hands-On Demonstrations

- Building a Webhook-Triggered Lead Enrichment Workflow with OpenAI
- Adding Error Handling and Retry Logic to an N8N Workflow
- Scheduling a Daily Competitor Monitoring Workflow with Cron and LLM

Skills Covered

- N8N
- Visual Workflow Automation
- Webhook Triggers
- LLM Node Integration
- Error Handling
- Cron Scheduling
- N8N + LangGraph

Module 16: Guardrails: NeMo, Guardrails AI & Safety

Topics Covered

- The AI safety landscape: why guardrails are non-negotiable in production
- OWASP LLM Top 10: understanding the most critical risks
- Prompt injection attacks: anatomy, real examples, and detection strategies
- NVIDIA NeMo Guardrails: architecture, Colang language, and dialog flows
- Writing Colang rails: defining allowed and blocked topics
- NeMo input rails: validating user messages before the LLM sees them
- NeMo output rails: filtering and validating LLM responses before delivery
- Guardrails AI framework: validators, the Hub, and the Guard object
- Built-in Guardrails AI validators: toxic language, secrets, competitor mentions
- Writing custom Guardrails AI validators for domain-specific rules
- PII detection with Microsoft Presidio: entities, recognisers, and anonymisation
- Building an end-to-end safety stack: input rail, LLM, output validator, PII scrub
- Toxicity, bias, and hallucination detection tools and integrations
- Constitutional AI principles: embedding safety constraints in agent design
- Testing the safety stack: red-teaming with adversarial prompt suites

Hands-On Demonstrations

- Writing NeMo Colang Rails to Block Off-Topic and Harmful Requests
- Adding PII Detection and Redaction with Presidio and Guardrails AI
- Red-Teaming an Agent with Prompt Injections and Hardening the Defences

Skills Covered

- NeMo Guardrails
- Colang
- Guardrails AI

- Presidio
- Prompt Injection Defence
- AI Safety Design
- OWASP LLM Top 10

Module 17: Monitoring, Evaluation & Fine-Tuning

Topics Covered

- The three pillars of LLM observability: logs, metrics, and distributed traces
- OpenTelemetry for AI: instrumenting LangGraph agents with spans and attributes
- Exporting traces to Jaeger and interpreting the trace waterfall
- Prometheus: scrape targets, metric types, and PromQL query basics
- Grafana: building dashboards for latency, token cost, and error rate
- Arize AI and Phoenix: LLM-specific monitoring, embedding drift, and data quality
- Cost tracking per run: token budgets, alerts, and quota management
- SLA and SLO design for agent systems: what to measure and why
- Fine-tuning fundamentals: when prompting is insufficient and fine-tuning is warranted
- LoRA (Low-Rank Adaptation): intuition, hyperparameters, and implementation
- QLoRA: quantised fine-tuning for resource-constrained environments
- Building a fine-tuning dataset from LangSmith agent traces
- Training a LoRA adapter on a custom domain dataset with HuggingFace PEFT
- Evaluating a fine-tuned model vs the base model on a held-out test set
- RLHF and DPO: overview, concepts, and when to consider them

Hands-On Demonstrations

- Instrumenting a LangGraph Agent with OpenTelemetry and Building a Grafana Dashboard
- Detecting Embedding Drift in a RAG Pipeline with Arize Phoenix
- Fine-Tuning a Model with LoRA and Benchmarking Against the Base Model

Skills Covered

- OpenTelemetry
- Prometheus
- Grafana
- Arize AI / Phoenix
- LLM Observability
- LoRA
- QLoRA
- HuggingFace PEFT
- SLA/SLO Design

Module 18: Dockerizing & Deploying AI Agents

Topics Covered

- Docker fundamentals: images, containers, layers, and the build cache
- Writing a production Dockerfile for a Python FastAPI + LangChain application
- Multi-stage builds: separating build-time dependencies from the runtime image
- Docker Compose: defining multi-service stacks with service, network, and volume configs
- Composing a full agent stack: FastAPI, ChromaDB, Redis, and a worker service
- Environment variable management: .env files, Docker secrets, and runtime injection
- Container health checks: liveness, readiness, and graceful shutdown patterns
- Pushing images to a container registry: Docker Hub and GCP Artifact Registry
- Deploying a containerised agent to GCP Cloud Run with auto-scaling
- Deploying to AWS ECS with Fargate: task definitions and service configuration
- GitHub Actions CI/CD: build, test, push, and deploy pipeline for AI services
- Blue/green deployments: switching traffic between versions with zero downtime
- Canary deployments: gradually rolling out a new agent version

- Monitoring deployed containers: Cloud Run logs, metrics, and alerting

Hands-On Demonstrations

- Building and Running a Multi-Service Agent Stack with Docker Compose
- Writing a GitHub Actions CI/CD Pipeline for an AI Agent Service
- Deploying a Containerised Agent to GCP Cloud Run with Zero Downtime
- Implementing a Blue/Green Deployment Switch for an Agent Service

Skills Covered

- Docker
- Docker Compose
- GitHub Actions
- GCP Cloud Run
- AWS ECS
- CI/CD
- Blue/Green Deployment
- Container Security

Module 19: Capstone Part 1: Architecture, Build & Integration

Topics Covered

- Capstone problem selection: choosing a real-world industry use case to solve
- System architecture design: identifying components, data flows, and boundaries
- Technology selection: reasoning through LangGraph vs CrewAI vs hybrid approaches
- Designing the RAG layer: index strategy, chunking method, and retrieval approach
- Designing the multi-agent layer: agent roles, task decomposition, and delegation flow
- Integrating MCP servers: identifying which external tools the system needs
- Wiring guardrails into the architecture: input rails, output validators, PII scrubbing

- Building the core agent loop: RAG + LangGraph + tool use running end-to-end
- Connecting the front-end: Streamlit or Gradio UI wired to the FastAPI agent backend
- Instrumenting with LangSmith tracing across all components
- Adding Grafana monitoring: latency, token cost, and error rate dashboards
- Peer design review: presenting the architecture and receiving structured feedback
- Iterating on the build based on review feedback before integration testing

Hands-On Demonstrations

- Architecting the Full Capstone System and Presenting the Design
- Building the Core Agent Loop with RAG, Multi-Agent Orchestration, and Guardrails
- Wiring LangSmith Tracing and Grafana Monitoring into the Full System
- Running End-to-End Integration Tests Across All Components

Skills Covered

- System Architecture Design
- RAG Layer Design
- Multi-Agent Design
- MCP Integration
- Guardrails Integration
- LangSmith Tracing
- Grafana Monitoring

Module 20: Capstone Part 2: Deployment

Topics Covered

- Dockerising all capstone services: writing production Dockerfiles for each component
- Writing the Docker Compose stack: linking FastAPI, ChromaDB, Redis, and the agent worker
- CI/CD pipeline for the capstone: automated testing, build, and cloud deployment

- Deploying the full stack
- Performance testing: measuring latency, throughput, and cost per query under load
- Security review: checking for prompt injection vulnerabilities, data leakage, and access control gaps
- Writing technical documentation: README, API spec, and architecture decision records
- Writing a project brief: summarising the problem, approach, results, and trade-offs
- Peer code review: reviewing a peer's codebase and providing structured written feedback
- Incorporating peer review feedback and shipping the final version
- Preparing the live demo narrative: telling the story of the system to a non-technical audience
- Live demo delivery: presenting the working system with monitoring dashboard live
- Capstone retrospective: what worked, what you would do differently, and key takeaways

Hands-On Demonstrations

- Deploying the Full Capstone Stack to Cloud with CI/CD
- Running a Performance and Security Audit on the Live System
- Delivering the Live System Demo with Real-Time Monitoring

Skills Covered

- Docker & Compose
- CI/CD Deployment
- Cloud Deployment
- Performance Testing
- Security Review
- Technical Documentation
- Live Demo Delivery

Module 21: Vibe Coding for Developers (Self-paced)

Topics:

- Vibe Coding Fundamentals
- Vibe Coding Tools Overview
- GitHub Copilot Setup and Features
- Chat and Interactive Coding
- Code Quality and Testing
- Advanced Copilot Techniques
- Cursor Fundamentals
- Codebase Intelligence
- Composer
- Development with Cursor

Hands-on

- Working with GitHub Copilot
- Building Projects with Cursor

Skills you will learn:

- Vibe Coding
- GitHub Copilot
- Cursor AI

Module 22: Programmatic Prompting with DSPy (Self-paced)

Topics:

- Introduction to DSPy
- DSPy Modules and Signatures
- ChainOfThought and Predict Modules
- ReAct Modules in DSPy

- DSPy Optimizers (BootstrapFewShot, MIPRO)
- Compiling and Validating DSPy Programs
- Metric Functions and Evaluation
- DSPy with Multiple LLMs
- Building Self-Improving Pipelines
- DSPy vs Manual Prompting Trade-offs
- DSPy Production Deployment

Hands-on

- Build a Self-Improving Prompt Optimization System with DSPy

Skills you will learn:

- DSPy framework
- Programmatic prompting
- Prompt optimization
- Self-improving systems

Module 23: No-Code Workflow Automation with Zapier and Make (Self-paced)

Topics:

- Zapier Fundamentals and Workflows
- Trigger and Action Patterns
- Multi-Step Zaps for Complex Logic
- Zapier API for Custom Integrations
- Zapier Tables for Agent Data Storage
- Make (Integromat) Overview and Modules
- Make Scenarios and Advanced Routing

- Webhooks and API Integration in Make
- Connecting Python Agents to Zapier/Make
- Scaling No-Code Automation

Hands-on

- Working with Zapier and Make

Skills you will learn:

- Zapier automation
- Make workflows
- No-code integrations
- Hybrid agent-automation

Module 24: Agent Interoperability: ACP, ANP & A2A Protocols (Self-paced)

Topics:

- Agent Interoperability Challenge
- Agent-to-Agent (A2A) Protocol Overview
- A2A SDK and Implementation
- Agent Communication Protocol (ACP) by IBM BeeAI
- Agent Network Protocol (ANP) with DIDs
- Agent Cards for Capability Discovery
- Cross-Framework Agent Communication
- Building Multi-Protocol Gateways
- Security and Trust in Agent Networks
- Enterprise Agent Federation
- Future Standards and Governance

Hands-on

- Build an Interoperable Multi-Framework Agent Network

Skills you will learn:

- Agent protocols
- Cross-framework interop
- Agent networks
- Protocol governance

Module 25: Generative AI and LLM Security

Topics:

- Threats in Generative AI Systems
- Common Attack Vectors in Generative AI Systems
- Model Theft and Extraction Attacks
- Mitigation Strategies for GenAI Risks
- LLM-Specific Threats and Risks
- Aligning LLM Output to Security Objectives
- Securing AI Training Data and Pipelines
- Risks in AI Model Hubs and Repositories
- Dependency Scanning and Third-Party Model Risks
- Bias, Fairness, and Ethical Design in AI Systems
- Regulatory and Compliance Standards
- Multimodal AI Threat Intelligence
- Defending Cyber Operations with Agentic AI

Hands-on

- Detecting Prompt Injection and Jailbreak Risks
- LLM Integration with Gemini API
- Securing AI Data Against Poisoning Risks
- Tracking Model Provenance and Scanning Dependencies
- Ethical Screening using Sola Security
- Agentic AI for Cybersecurity Triage

Skills you will learn:

- GenAI Security & Threat Analysis
- LLM Risk Assessment & Mitigation
- AI Pipeline & Data Security
- Ethical AI & Compliance Implementation